TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN – ĐHQG-HCM
KHOA KỸ THUẬT MÁY TÍNH

IT012 – TỔ CHỨC VÀ CẤU TRÚC MÁY TÍNH II

LAB 01

CỔNG LUẬN LÝ VÀ ĐẠI SỐ BOOLE TRÊN LOGISIM

Họ và tên	Nguyễn Vũ Thịnh	**MSSV**	25521778
Lớp	IT012.R12.CNVN.2	**Ngày thực hành**	29/09/2026
Tự chấm	9.25	**Link git**	x

1. Tổng quan buổi thực hành

Sinh viên dùng phần mềm **Logisim** để biến kiến thức Chương 3 (đại số Boole, cổng luận lý, tối ưu luận lý, bìa Karnaugh) thành các thí nghiệm **dự đoán – dựng mạch – quan sát – giải thích**.

Sau buổi thực hành, Sinh viên có thể:

- Dựng mạch từ các cổng luận lý trong Logisim, đọc và ghi được bảng chân trị của mạch.
- Phân biệt đẳng thức Boole **đúng/sai** bằng mô phỏng, tìm phản ví dụ, chứng minh bằng tiên đề/định lý.
- Lập bảng chân trị từ một đặc tả, viết dạng chính tắc (Σm, ΠM) và hiện thực thành mạch.
- Tối ưu biểu thức bằng đại số Boole và bìa Karnaugh (gom 1 → SoP, gom 0 → PoS), tính chi phí C(B), kiểm chứng tương đương.
- Đánh giá khi nào bìa Karnaugh không rút gọn được hàm và cần chọn loại cổng phù hợp hơn.

Phần	Nội dung	Thời gian gợi ý	Điểm
Phần 0	Làm quen Logisim, kiểm tra môi trường	20 phút	
TN 1	Các cổng luận lý: dự đoán – quan sát – giải thích	30 phút	1,5
TN 2	Đẳng thức Boole: đúng hay sai?	40 phút	2,0
TN 3	Từ đặc tả → bảng chân trị → dạng chính tắc → mạch	45 phút	2,0
TN 4	Tối ưu luận lý: đại số Boole và bìa Karnaugh (SoP / PoS)	55 phút	4,0
TN 5	Mở rộng: khi bìa Karnaugh "bó tay" (Mở rộng)	20 phút	2,0
Kết	Tổng kết, phản tư, nộp bài	15 phút	0,5
Tổng		**225 phút**	**12,0/10,0**

Quy ước làm bài

- **Ô nền vàng nhạt** là chỗ Sinh viên điền hoặc trả lời trực tiếp vào file này (gõ chữ, hoặc dán ảnh chụp màn hình vào khung nét đứt).
- Ký hiệu: NOT là dấu phẩy trên (A'); AND viết liền hoặc A·B; OR là A + B; XOR là A ⊕ B. Biến đầu tiên là bit trọng số cao nhất (ABCD: A là MSB).
- Mỗi thí nghiệm theo chu trình **DỰ ĐOÁN → QUAN SÁT → GIẢI THÍCH**. Ghi dự đoán **trước** khi mô phỏng; dự đoán sai **không bị trừ điểm**, phần giải thích vì sao sai mới được chấm.

PHẦN 0 — Làm quen Logisim (20 phút)

Giảng viên hướng dẫn nhanh giao diện Logisim theo tài liệu *Hướng dẫn sử dụng Logisim*. Sinh viên tự đánh dấu ☑ khi thực hiện được từng kỹ năng dưới đây (bỏ qua nếu đã thành thạo).

☑	Kỹ năng	Gợi ý
☑	Tạo project mới; đặt tên circuit đầu tiên là **Warmup**; lưu file **IT012_Lab01_<MSSV>.circ**	Mỗi thí nghiệm dùng một circuit riêng trong **cùng một file** (Project ▸ Add Circuit).
☑	Đặt **Input Pin** và **Output Pin**; đặt nhãn (Label) cho từng pin	Nhãn ngắn gọn: A, B, C, F… Công cụ phân tích mạch lấy tên biến từ nhãn.
☑	Đặt cổng AND, OR, NOT; đổi **Number of Inputs**	Chọn cổng rồi chỉnh thuộc tính ở ngăn thuộc tính (góc dưới bên trái).
☑	Nối dây; rẽ nhánh từ một dây đã có	Dây chỉ cắt ngang nhau thì **không** nối. Chỗ nối thật có chấm tròn.
☑	Dùng **Poke tool** đổi giá trị ngõ vào; đọc màu dây	Bấm lên Input Pin để đổi 0 ↔ 1.
☐	Thêm circuit mới và chuyển qua lại giữa các circuit	Tên circuit dùng đúng quy ước: TN1, TN2, TN3, TN4, TN5.
☑	Mở **Project ▸ Analyze Circuit** và xem tab **Table**	Công cụ tự sinh bảng chân trị của mạch. Sẽ dùng nhiều lần trong buổi này.
☑	Chụp màn hình mạch và dán vào khung ảnh của file này	Chụp cả mạch lẫn nhãn ngõ vào/ngõ ra.

Phiên bản Logisim đang dùng	4.1.0

Đọc màu dây trong Logisim

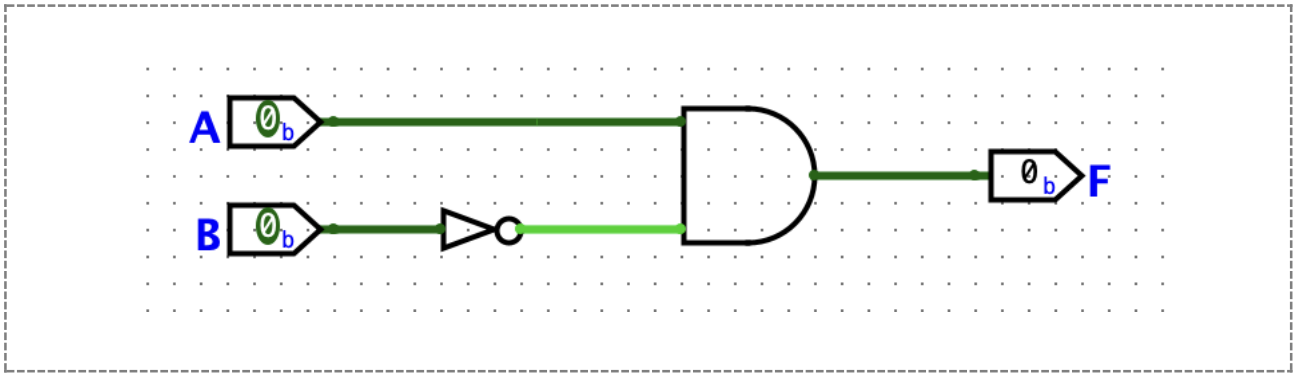
Xanh lá sáng = 1 · **Xanh lá đậm** = 0 · **Xanh dương** = dây chưa được điều khiển (chưa nối tới nguồn) · **Đỏ** = lỗi/xung đột.

Nếu ngõ ra không đổi khi Sinh viên bật/tắt ngõ vào, hãy kiểm tra xem dây đã thật sự được nối hay chỉ "nằm cạnh nhau".

Bước 0 – Kiểm tra nhanh. Trong circuit **Warmup**, dựng mạch **F = A·B'** (1 cổng NOT + 1 cổng AND), đặt nhãn A, B, F. Dùng Analyze Circuit ▸ Table để điền bảng chân trị.

A	B	F = A·B'
0	0	0
0	1	0
1	0	1
1	1	0

Circuit Warmup chỉ để làm quen, không nộp.



THÍ NGHIỆM 1 — Các cổng luận lý: dự đoán, quan sát, giải thích

Thời lượng gợi ý: 30 phút · Circuit: **TN1** · Điểm: 1,5

A. Cổng luận lý 2 ngõ vào

Bước 1.1 – Dựng mạch. Trong circuit **TN1**, đặt 2 Input Pin **A**, **B** và 7 Output Pin có nhãn. Nối: NOT (chỉ ngõ vào A), rồi AND, OR, NAND, NOR, XOR, XNOR (mỗi cổng nhận A và B).

QUAN SÁT Dùng Poke tool đổi A, B và ghi các ngõ ra vào Bảng 1.

Bảng 1. Bảng chân trị các cổng 2 ngõ vào (đo trên Logisim)

A	B	A'	AND	OR	NAND	NOR	XOR	XNOR
0	0	1	0	0	1	1	0	1
0	1	1	0	1	1	0	1	0
1	0	0	0	1	1	0	1	0
1	1	0	1	1	0	0	0	1

Câu 1.1 (Phân tích). Từ Bảng 1, chỉ ra các cặp cổng luôn cho ngõ ra **đối nghịch nhau** ở mọi tổ hợp. Viết mỗi quan hệ dưới dạng biểu thức Boole (ví dụ: X = Y').

Trả lời: A=A'; AND=NAND'; OR=NOR'; XOR = XNOR'.

Câu 1.2 (Phân tích). Một số cổng có ngõ ra khác với ba hàng còn lại ở đúng **một** hàng. Điền vào bảng (tham khảo bảng minterm/maxterm ở slide *Dạng chính tắc*).

Đặc điểm	Cổng	Hàng (A B)	Minterm / Maxterm tương ứng
Ngõ ra = 1 ở đúng một hàng	AND	1 1	$m_3 = AB$
Ngõ ra = 1 ở đúng một hàng	NOR	0 0	$m_0 = A'B'$
Ngõ ra = 0 ở đúng một hàng	OR	0 0	$M_0 = A + B$
Ngõ ra = 0 ở đúng một hàng	NAND	1 1	$M_0 = A' + B'$

Câu 1.3 (Hiểu). XNOR(A, B) bằng 1 khi nào? Từ đó giải thích vì sao cổng XNOR có thể dùng làm **mạch so sánh** hai tín hiệu (sẽ dùng ở Thí nghiệm 2).

Trả lời:
- *XNOR = (XOR)', mà XOR bằng 1 khi số ngõ vào bằng 1 là số lẻ. Vì vậy XNOR bằng 1 khi số ngõ vào bằng 1 là số chẵn. Tức là XNOR(A,B) = 1 khi A = B = {0,1};*
- *Lý do XNOR có thể dùng làm mạch so sánh 2 tín hiệu: Vì khi 2 tín hiệu đó bằng nhau (cùng bằng 0 hoặc 1) thì XNOR sẽ trả về giá trị true, nếu khác nhau sẽ trả về false. Điều đó giúp so sánh 2 tín hiệu*

B. Cổng nhiều ngõ vào

Bước 1.2 – Dựng thêm. Thêm Input Pin **C**. Trong cùng circuit TN1, dựng: (a) cổng **XOR** với Number of Inputs = 3 (ngõ ra **XOR3**); (b) cổng **NAND** với Number of Inputs = 3 (ngõ ra **NAND3**); (c) hai cổng NAND 2 ngõ vào nối tiếp: X = NAND(A, B), rồi Y = NAND(X, C) (ngõ ra **CHAIN**).

DỰ ĐOÁN **Trước khi** mô phỏng, dựa vào định nghĩa XOR ở slide (ngõ ra = 1 khi *số ngõ vào bằng 1 là lẻ*), điền cột "XOR3 – dự đoán" của Bảng 2.

QUAN SÁT Sau đó điền 3 cột còn lại bằng giá trị đo được.

Bảng 2. Cổng 3 ngõ vào

A	B	C	XOR3 – dự đoán	XOR3 – đo được	CHAIN = (A NAND B) NAND C	NAND3
0	0	0	0	0	1	1
0	0	1	1	1	0	1
0	1	0	1	1	1	1
0	1	1	0	0	0	1
1	0	0	1	1	1	1
1	0	1	0	0	0	1
1	1	0	0	0	1	1
1	1	1	1	1	1	0

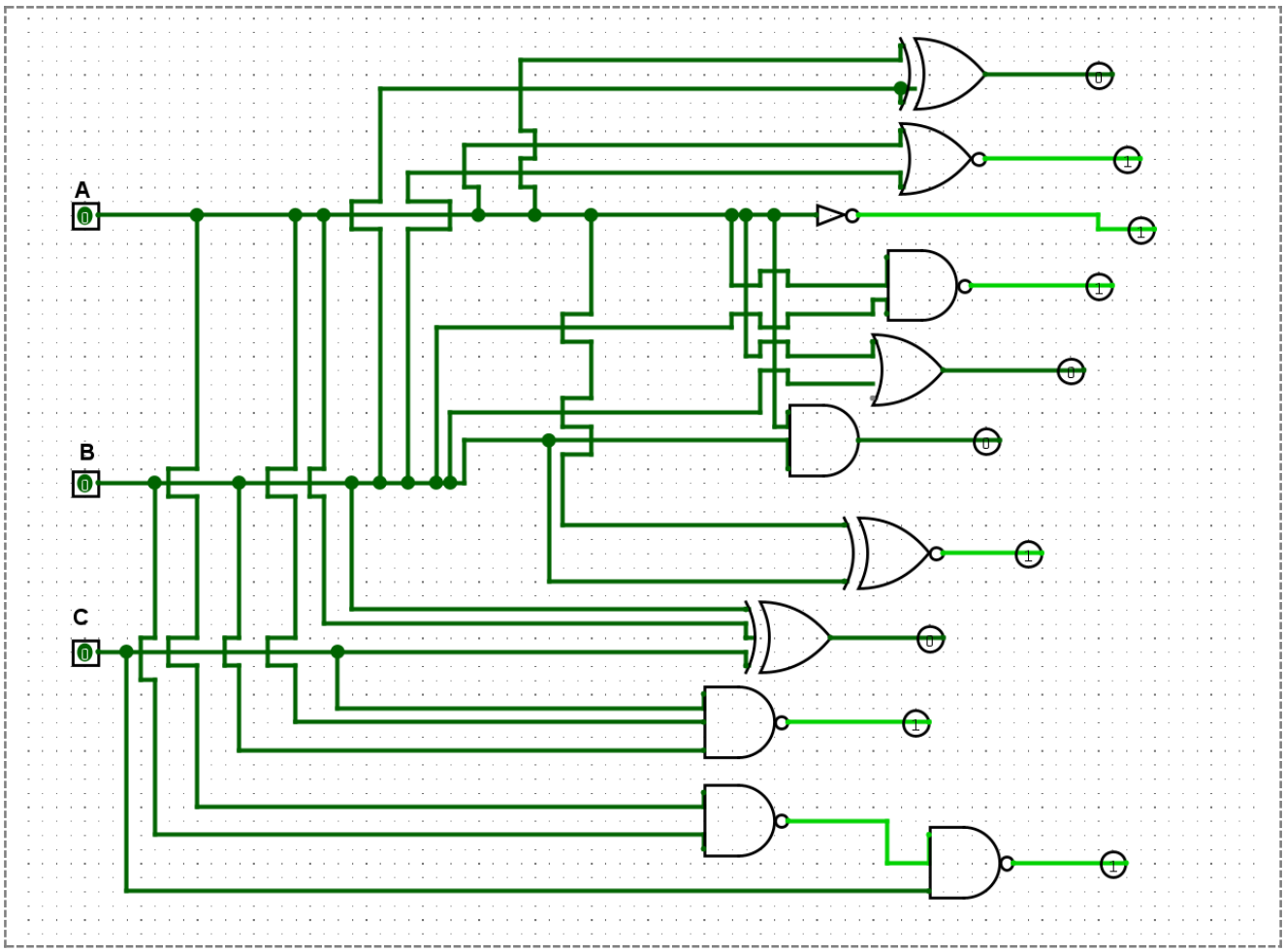
Câu 1.4 (Phân tích). So sánh hai cột XOR3 và ghi các hàng khác nhau (nếu có). Mở thuộc tính **Multiple-Input Behavior** của cổng XOR (nếu phiên bản Logisim của Sinh viên có), thử từng giá trị: giá trị nào cho kết quả khớp định nghĩa ở slide? Giá trị còn lại thực chất đang mô tả điều kiện gì?

Trả lời:
- *Hai cột XOR3 đều có giá trị giống nhau ở các hàng*
- *Chế độ "When an odd number are on": Tất cả giá trị đều có kết quả khớp với định nghĩa ở Slide*
- *Chế độ "When one input is on": Đầu ra chỉ bằng 1 khi chỉ có duy nhất 1 ngõ vào bằng 1, giá trị này mô tả điều kiện chỉ có duy nhất một giá trị đầu vào được phép đúng*

Câu 1.5 (Áp dụng). Cột CHAIN có trùng cột NAND3 không? Điền các bước biến đổi CHAIN thành biểu thức đơn giản (dùng định lý De Morgan và x'' = x), rồi cho biết phép NAND **thiếu** tính chất nào so với AND/OR.

CHAIN = ((A·B)' · C)'	=	*((A'+B').C)'*	=	AB+C'

Tính chất NAND bị thiếu và ví dụ minh hoạ: Tính chất mà NAND bị thiếu chính là tính kết hợp. Ví dụ minh họa qua tình huống ở Bảng 2. Rõ ràng (A NAND B) NAND C và NAND3 đã trả về những kết quả khác nhau ở nhiều trường hợp, trong khi đó (A AND B) AND C = AND3, (A OR B) OR C = OR3 ở mọi trường hợp


A
B
C

THÍ NGHIỆM 2 — Đẳng thức Boole: đúng hay sai?

Thời lượng gợi ý: 40 phút · Circuit: **TN2** · Điểm: 2,0

> **Tình huống**
> Một bạn cùng lớp đưa ra 5 "đẳng thức Boole" và bảo rằng "nhìn qua thấy đúng". Sinh viên sẽ dùng Logisim để **kiểm chứng bằng vét cạn** (thử mọi tổ hợp ngõ vào), tìm phản ví dụ nếu có, rồi **chứng minh** bằng tiên đề/định lý.

DỰ ĐOÁN Chỉ bằng trực giác và kiến thức từ slide, hãy đánh giá từng đẳng thức là Đúng (Đ) hay Sai (S) **trước khi** dựng mạch.

Bảng 3. Các đẳng thức cần kiểm chứng (x, y, z là biến luận lý)

#	Đẳng thức	Dự đoán (Đ/S)	Kết quả mô phỏng (Đ/S)
1	(x + y)' = x'·y'	Đ	Đ
2	(x·y)' = x'·y'	S	S
3	x + y·z = (x + y)·(x + z)	Đ	Đ
4	x + x'·y = x	S	S
5	x + x'·y = x + y	Đ	Đ

DỰNG MẠCH Trong circuit **TN2**: đặt 3 Input Pin **x, y, z**. Với mỗi đẳng thức #i, dựng **vế trái (L)** và **vế phải (R)**, rồi đưa cả hai vào một cổng **XNOR** để có ngõ ra **EQi**. Vì XNOR = 1 khi hai ngõ vào bằng nhau (Câu 1.3), EQi = 1 nghĩa là hai vế bằng nhau tại tổ hợp đó. Đặt 5 Output Pin nhãn EQ1…EQ5.

Gợi ý: dùng chung các cổng NOT của x và y, và những biểu thức con giống nhau (ví dụ x'·y' xuất hiện ở #1 và #2; x + x'·y xuất hiện ở #4 và #5). Toàn mạch chỉ cần khoảng 20 cổng.

KIỂM CHỨNG Mở **Project ▸ Analyze Circuit ▸ Table** và chép kết quả vào Bảng 4. Một đẳng thức là đúng khi cột EQ tương ứng bằng 1 ở **cả 8 hàng**.

Bảng 4. Ngõ ra EQ của mạch TN2

x	y	z	EQ1	EQ2	EQ3	EQ4	EQ5
0	0	0	1	1	1	1	1
0	0	1	1	1	1	1	1
0	1	0	1	0	1	0	1
0	1	1	1	0	1	0	1
1	0	0	1	0	1	1	1
1	0	1	1	0	1	1	1
1	1	0	1	1	1	1	1
1	1	1	1	1	1	1	1

GIẢI THÍCH Hoàn thành Bảng 5. Với đẳng thức sai, chọn **một** hàng có EQ = 0, tự tính giá trị vế trái và vế phải **bằng tay** để xác nhận phản ví dụ.

Bảng 5. Kết luận và phản ví dụ

#	Kết luận (Đ/S)	Nếu sai: phản ví dụ (x y z) và giá trị vế trái / vế phải	Nếu sai: sửa lại thành đẳng thức đúng
1	Đ	X	X
2	S	x=0,y=1: VT = (0.1)'=1 VP =1.0 = 0 VT ≠ VP -> Sai	(x.y)'=x'+y'
3	Đ	X	X
4	S	x=0,y=1: VT = 0+1.1=1 VP=0 VT ≠ VP -> Sai	x + x'y = x + y
5	Đ	X	X

Câu 2.1 (Phân tích). Đẳng thức #4 sai còn #5 đúng, dù chỉ khác nhau một ký hiệu. So sánh #4 với định lý hấp thụ x + x·y = x ở slide và giải thích vì sao dễ nhầm lẫn giữa hai trường hợp này.

Trả lời:

- So sánh (#4) với định lý hấp thụ x + x.y = x ở slide:
  + Dùng luật phân phối cho định lý hấp thụ ta được: x+xy=(x+x)(x+y)=x(x+y)=x+xy=x(1+y)=x
  + Dùng luật phân phối cho (#4) ta được: x + x'y=(x+x')(x+y)=1(x+y)=x+y.
  ➔ Dễ thấy rằng chỉ khác ở x'y và xy mà lại ra kết quả khác nhau nằm ở chỗ sau khi phân phối, ta thu được (x+x) = x và (x+x') = 1. Dẫn đến dễ nhầm lẫn giữa 2 trường hợp. Giải pháp là nên thực hiện 1 bước nhẩm phân phối để chắc chắn hơn.

Câu 2.2 (Áp dụng). Đẳng thức #3 tương ứng tiên đề nào ở slide? Viết **đẳng thức đối ngẫu** của nó và cho biết đẳng thức đó đúng hay sai (dựa vào nguyên lý đối ngẫu ở slide).

Trả lời:

- *Đẳng thức #3 ứng với tiên đề 4 – Phân phối.*
- *Đẳng thức đối ngẫu: x.(y+z)=(x.y)+(x.z) (1).*
- *Đẳng thức (1) đúng vì: Theo nguyên lý đối ngẫu, đẳng thức gốc #3 là một tiên đề đúng, nên đẳng thức đối ngẫu của nó là (1) cũng đúng.*

Từ kiểm chứng đến chứng minh

CHỨNG MINH **(a)** Chứng minh đẳng thức #5 bằng tiên đề/định lý. Điền tên tiên đề/định lý đã dùng ở mỗi bước.

Biểu thức	Vì (tiên đề / định lý nào?)
x + x'·y	(bắt đầu)
= (x + x')·(x + y)	Tiên đề Phân phối
= 1·(x + y)	Tiên đề Phần tử bù
= x + y	Tiên đề Phần tử trung hòa

CHỨNG MINH **(b)** Đơn giản F(A, B, C) = ABC + A'B + ABC' (bài tập ở slide) bằng đại số Boole. Ghi từng bước và quy tắc áp dụng.

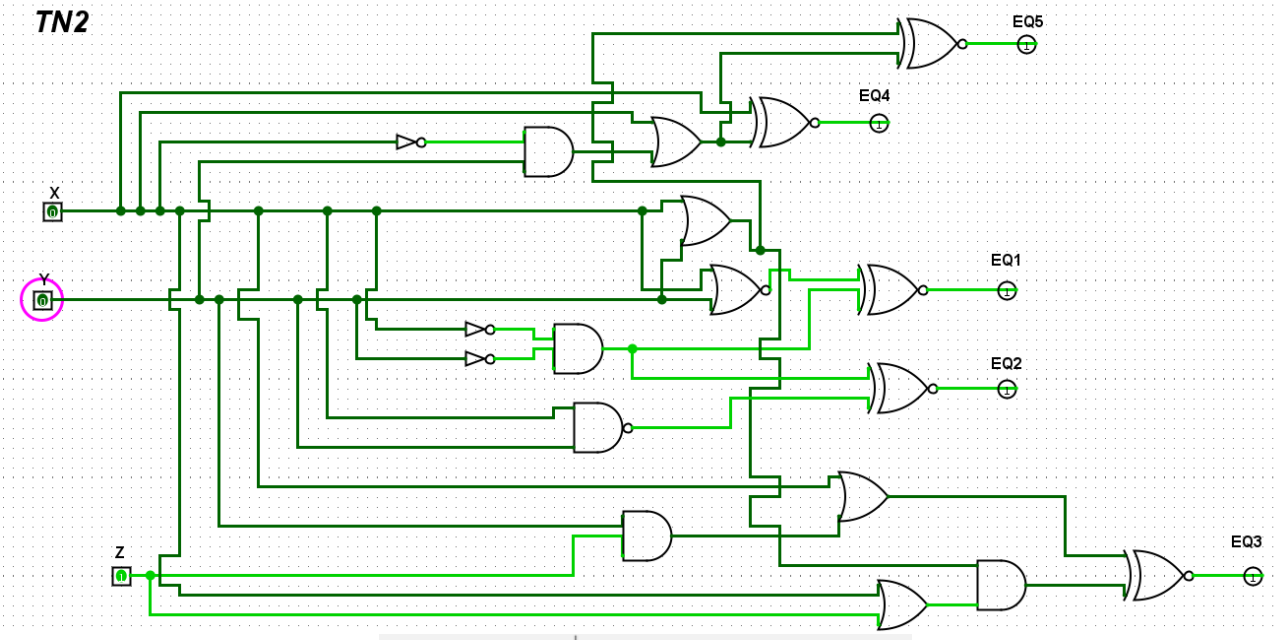
Bước	Biểu thức	Quy tắc áp dụng
0	F = ABC + A'B + ABC'	(đề bài)
1	F=AB(C+C')+A'B	Luật Giao hoán + Luật Phân phối
2	F=AB.1+A'B	Tiên đề Phần tử bù
3	F=AB+A'B	Tiên đề Phần tử trung hòa
4	F=B(A+A')	Tiên đề Phân Phối
5	F= B.1	Tiên đề Phần tử bù

Kết quả rút gọn: F =	B

Câu 2.3 (Đánh giá). Kiểm chứng bằng bảng chân trị cần 2^n hàng cho n biến. Với một đẳng thức 10 biến cần bao nhiêu hàng? Khi nào Sinh viên tin vào kiểm chứng bằng mô phỏng, khi nào cần chứng minh đại số?

Trả lời:

- Vì sao bảng chân trị cần 2^n hàng cho n biến:
  + Vì 1 biến nhị phân được định nghĩa chỉ nhận được 2 giá trị {0,1}.
  + Theo quy tắc đếm ta có như sau: mỗi biến $n_1, n_2, n_3, \dots, n_n$ có 2 khả năng nhận giá trị, một là nhận 1, hai là nhận 0. Vì vậy ta có tổng số tổ hợp là: 2.2.2.2.2.2.2,...n = 2^n
  + Kiểm chứng: bảng chân trị có:
    2 biến: $2^2 = 4$ có 4 hàng gồm {00,01,10,11}
    3 biến $2^3 = 8$ có 8 hàng gồm {000,001,010,011,100,101,110,111}
    ...
  + Với 1 đẳng thức 10 biến cần $2^{10} = 1024$ hàng
- Tin vào kiểm chứng bằng mô phỏng khi thực nghiệm trên những kết quả có số lượng biến nhỏ đến trung bình vì máy tính hoàn toàn có khả năng thực hiện cực kỳ nhanh – chính xác trong giới hạn kiểm tra vét cạn. Tuy nhiên, khi số lượng biến lớn đến cực lớn, việc dùng máy tính để mô phỏng sẽ gặp sự bùng nổ tổ hợp, dẫn đến máy tính tính trong thời gian quá dài. Ví dụ thiết kế 1 mạch có 64 biến gồm 2^{64} tổ hợp, không máy tính nào tính được trong thời gian thực. Vì vậy cần dùng sự chứng minh đại số chắc chẽ đếm đảm bảo logic này đúng đắn trong thực tế, và đúng cho trường hợp tổng quát (Tức với mọi số lượng biến).



a	b	c	x	y	z	u	v
0	0	0	1	1	1	1	1
0	0	1	1	1	1	1	1
0	1	0	1	0	1	0	1
0	1	1	1	0	1	0	1
1	0	0	1	1	1	0	1
1	0	1	1	1	1	0	1
1	1	0	1	1	1	1	1
1	1	1	1	1	1	1	1

THÍ NGHIỆM 3 — Từ đặc tả đến dạng chính tắc và mạch

Thời lượng gợi ý: 45 phút · Circuit: **TN3** · Điểm: 2,0

> **Bài toán "Bộ lọc kênh"**
> Một hệ thống có 16 kênh, đánh số N = 0…15 và truyền đi bằng mã nhị phân 4 bit **ABCD** (A là MSB): N = 2^3A + 2^2B + 2^1C + 2^0D. Chỉ những kênh có số hiệu là **số nguyên tố** được phép kích hoạt. Mạch cần thiết kế có 4 ngõ vào A, B, C, D và ngõ ra **F = 1** khi N là số nguyên tố.
> *Số nguyên tố: số tự nhiên lớn hơn 1, chỉ chia hết cho 1 và chính nó.*

Bước 3.1 – Lập bảng chân trị. Điền cột F cho từng giá trị N.

Bảng 6. Bảng chân trị F(A, B, C, D)

N	A	B	C	D	F
0	0	0	0	0	0
1	0	0	0	1	0
2	0	0	1	0	1
3	0	0	1	1	1
4	0	1	0	0	0
5	0	1	0	1	1
6	0	1	1	0	0
7	0	1	1	1	1

N	A	B	C	D	F
8	1	0	0	0	0
9	1	0	0	1	0
10	1	0	1	0	0
11	1	0	1	1	1
12	1	1	0	0	0
13	1	1	0	1	1
14	1	1	1	0	0
15	1	1	1	1	0

Câu 3.1 (Hiểu). Sinh viên xếp N = 0 và N = 1 vào loại nào (F = 0 hay F = 1)? Dựa vào phần nào của định nghĩa?

Trả lời:
- *Xếp N={0,1} vào loại F=0 vì theo định nghĩa, chỉ có số nguyên tố mới làm cho F=1*

Dạng chính tắc và chi phí

Bước 3.2 – Viết dạng chính tắc từ Bảng 6 (tham khảo ký hiệu minterm/maxterm ở slide).

SoP chính tắc: F = Σm(…). Các chỉ số minterm:	F(A,B,C,D)=m2+m3+m5+m7+m11+m13
PoS chính tắc: F = ΠM(…). Các chỉ số maxterm:	F(A,B,C,D)=M0.M1.M4.M6.M8.M9.M10.M12.M14.M15
Viết đầy đủ SoP chính tắc (các minterm theo A, B, C, D):	F(A,B,C,D)=A'B'CD'+A'B'CD+A'BC'D+A'BCD+AB'CD+ABC'D

Tính chi phí theo công thức C(B) = O(B) + ΣP_j(B) ở slide *Tối ưu luận lý*. Với PoS, áp dụng tương tự: đếm số thừa số tổng và số literal trong mỗi thừa số.

Dạng	Số term O(B)	Số literal mỗi term	C(B)
SoP chính tắc	6	4	30
PoS chính tắc	10	4	50

DỰ ĐOÁN Chưa vẽ bìa Karnaugh. Theo Sinh viên, sau khi **tối ưu tối đa**, dạng nào sẽ có chi phí thấp hơn? (Ghi lại để đối chiếu ở Thí nghiệm 4.)

☐ SoP rẻ hơn	☐ PoS rẻ hơn	☐ Bằng nhau	Lý do (ngắn):
Lý do: Vì ở bước 3.2, chi phí SoP chỉ có 30, trong khi PoS tận 50			

Hiện thực dạng SoP chính tắc trên Logisim

DỰNG MẠCH Trong circuit **TN3**, dựng F ở dạng SoP chính tắc từ Bước 3.2: 4 Input Pin **A, B, C, D**, các cổng NOT cần thiết, các cổng **AND 4 ngõ vào** (mỗi minterm một cổng), một cổng **OR** nhiều ngõ vào và 1 Output Pin **F**. Đổi Number of Inputs cho đúng.

Gợi ý bố trí: kéo 8 đường dọc ứng với A, A', B, B', C, C', D, D', rồi nối từng cổng AND vào đúng đường cần lấy. Cách này dễ kiểm tra và dễ sửa lỗi.

KIỂM CHỨNG Dùng Analyze Circuit ▸ Table, so sánh với Bảng 6 rồi hoàn thành Bảng 7. Nếu có hàng không khớp, ghi lại nguyên nhân Sinh viên tìm ra.

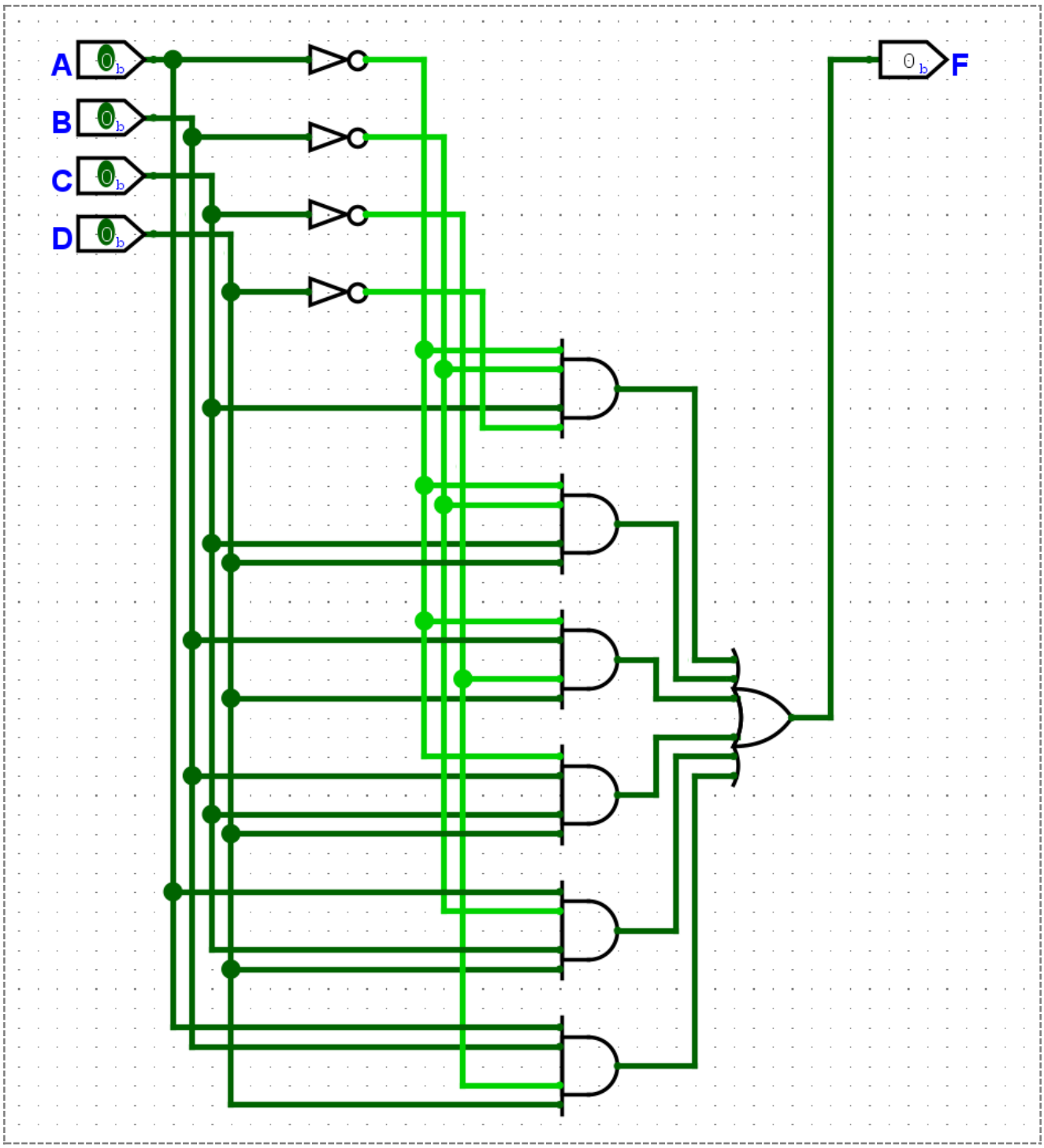
Bảng 7. Thống kê mạch SoP chính tắc

Số cổng AND (mỗi cổng mấy ngõ vào?)	6 cổng AND4
Số cổng OR (mấy ngõ vào?)	1 cổng OR6
Số cổng NOT	4 cổng NOT
Tổng số **chân vào** của các cổng AND và OR (không tính NOT)	30 chân vào
Số hàng của Analyze Circuit khớp với Bảng 6 (trên 16)	16/16
Lỗi đã gặp và cách phát hiện (nếu có)	X

Câu 3.2 (Phân tích). So sánh "tổng số chân vào của các cổng AND/OR" ở Bảng 7 với C(B) tính ở Bước 3.2. Sinh viên nhận xét gì về ý nghĩa của công thức chi phí? (Chú ý: một term chỉ có 1 literal thì tương ứng với gì trong mạch?)

Trả lời:

- Tổng số chân vào của các cổng AND/OR ở Bảng 7 bằng với C(B) tính ở Bước 3.2.
- *Nhận xét về ý nghĩa công thức chi phí:* Công thức tính chi phí có giá trị tính toán chính xác chi phí trước khi thực hiện xây dựng mạch. Trước khi xây dựng mạch từ biểu thức luận lý, ta dùng công thức tính chi phí để xác định trước chi phí xây dựng, từ đó xem mạch có khả thi để dựng không, chi phí thế này có bị quá lớn không. Từ đó suy nghĩ về cách tối ưu (nếu có/nếu đem lại giá trị thời gian/chi phí có lợi).
- Một term chỉ có 1 literal thì tương ứng với một dây duy nhất đi trực tiếp vào cổng, không cần cổng phụ là AND hay OR.


A
B
C
D
F

16 of 16 rows shown

A	B	C	D	F
0	0	0	0	0
0	0	0	1	0
0	0	1	0	1
0	0	1	1	1
0	1	0	0	0
0	1	0	1	1
0	1	1	0	0
0	1	1	1	1
1	0	0	0	0
1	0	0	1	0
1	0	1	0	0
1	0	1	1	1
1	1	0	0	0
1	1	0	1	1
1	1	1	0	0
1	1	1	1	0

THÍ NGHIỆM 4 — Tối ưu luận lý: đại số Boole và bìa Karnaugh

Thời lượng gợi ý: 55 phút · Circuit: **TN4** · Điểm: 3,0

A. Khởi động: bìa Karnaugh 3 biến

Bước 4.1. Vẽ bìa Karnaugh cho hàm F(A, B, C) = ABC + A'B + ABC' (hàm Sinh viên đã rút gọn bằng đại số ở Bước 2.5b). Điền giá trị F vào bìa, gom nhóm và so sánh với kết quả đại số.

A \ BC	00	01	11	10
0	0	0	1	1
1	0	0	1	1

Nhóm gom được (số ô, các minterm)	Gom được 1 minterm 4 ô (2,3,6,7)
Biểu thức sau khi gom	F = B
Chi phí C(B): trước khi rút gọn → sau khi rút gọn	Trước: 30, Sau: 0
Có trùng với kết quả đại số ở Bước 2.5b không?	Có

B. Tối ưu hàm "Bộ lọc kênh" bằng bìa Karnaugh 4 biến

Bước 4.2 – Điền bìa. Chuyển Bảng 6 sang bìa Karnaugh. Chú ý thứ tự **mã Gray** của hàng và cột (00, 01, 11, 10).

AB \ CD	00	01	11	10
00	0	1	3	2
01	4	5	7	6
11	12	13	15	14
10	8	9	11	10

Tự kiểm tra vị trí trước khi gom nhóm: ô ở hàng và cột dưới đây là minterm số mấy?

AB = 01, CD = 10 →	m_6	AB = 11, CD = 01 →	m_13	AB = 10, CD = 11 →	m_11

Bước 4.3 – Gom nhóm 1 (SoP). Mỗi nhóm là một term của biểu thức tối giản. Điền Bảng 8 (thừa hàng nào thì để trống).

Bảng 8. Các nhóm gom theo minterm

Nhóm	Các ô (m)	Số ô	Biến bị loại	Term
1	3,2	2	D	A'B'C
2	5,7	2	C	A'BD
3	5,13	2	A	BC'D
4	3,11	2	A	B'CD
5				
F_{SoP} =	A'B'C+A'BD+BC'D+B'CD		**C(B) =**	16

Câu 4.1 (Phân tích). Ô minterm nào chỉ có **một** cách gom nhóm? Ô nào có **nhiều hơn một** cách gom lớn nhất? Nếu chọn cách gom khác nhau cho cùng một ô, chi phí có thay đổi không? Điều đó nói gì về tính **duy nhất** của dạng tối giản?

Trả lời (ghi cả các biểu thức tối giản khác nhau nếu có):

- Ô chỉ có một cách gom nhóm: m_2, m_11, m_13
- Ô có nhiều hơn một cách gom lớn nhất: m_5, m_7, m_3
- Nếu chọn cách gom khác nhau cho cùng 1 ô với điều kiện nhóm đó vẫn giữ nguyên kích cỡ và tính hợp lệ, thì khi đó chi phí sẽ giữ nguyên. Ngược lại, nếu chọn cách gom nhóm nhỏ hơn cho cùng 1 ô (thay vì ô đó gom 2 thì chỉ gom 1), khi đó chi phí chắc chắn sẽ tăng lên. Điều đó nói rằng, tính duy nhất của dạng tối giản không phải cố định, 1 hàm Boole có thể có nhiều dạng tối giản khác nhau theo tùy cách gom, tuy nhiên về mặt chi phí C(B) tối giản của 1 hàm Boole hay 1 biểu thức luận lý chắc chắn là cố định, chỉ có 1 chi phí nhỏ nhất duy nhất.

Bước 4.4 – Gom nhóm 0 (PoS). Gom các ô có F = 0. Theo slide, với nhóm maxterm: biến mang giá trị **0** giữ nguyên, biến mang giá trị **1** lấy bù; các thừa số nối với nhau bằng phép AND.

Bảng 9. Các nhóm gom theo maxterm

Nhóm	Các ô (M)	Số ô	Biến bị loại	Thừa số tổng
1	M(0,1,8,9)	4	A, D	B+C
2	M(4,6,12,14)	4	A, C	B'+D
3	M(8,10,12,14)	4	B, C	A'+D
4	M(14,15)	2	D	A'+B'+C'
5				
F_{PoS} =	(B+C).(B'+D).(A'+D).(A'+B'+C')		**C(B) =**	13

Câu 4.2 (Hiểu). Có nhóm nào gồm các ô **không nằm sát nhau** trên hình vẽ (nằm ở hai mép đối diện của bìa)? Vì sao nhóm đó vẫn hợp lệ?

Trả lời:
- Các ô không nằm sát nhau trên hình vẽ nhưng vẫn hợp lệ là hoàn toàn đúng theo logic của đại số Boole, ta xét ví dụ sau:
  + Giả sử ta góm nhóm A'B'CD + AB'CD = B'CD(A'+A) = B'CD.1= B'CD.
  + Dễ thấy rằng biến A đã bị triệt tiêu, vì vậy nhóm này hoàn toàn hợp lệ vì đã đáp ứng được việc "loại bỏ bớt đi số literal khỏi term – tối ưu chi phí thiết kế".

C. Kiểm chứng và so sánh trên Logisim

DỰNG MẠCH Trong circuit **TN4**: sao chép (Ctrl+C / Ctrl+V) mạch SoP chính tắc từ TN3 để có ngõ ra **F_can**. Dựng thêm mạch **SoP tối giản** (ngõ ra **F_sop**) và mạch **PoS tối giản** (ngõ ra **F_pos**), dùng **chung** 4 Input Pin A, B, C, D và tận dụng các đường bù đã có. Khuyến khích thêm 2 cổng XNOR để có EQ1 = (F_can ≡ F_sop) và EQ2 = (F_can ≡ F_pos).

KIỂM CHỨNG Dùng Analyze Circuit ▸ Table và đếm số hàng mà các ngõ ra trùng nhau.

Số hàng (trên 16) mà F_can = F_sop	16/16
Số hàng (trên 16) mà F_can = F_pos	16/16
Nếu có hàng không trùng: nhóm/term nào bị sai, sửa thế nào?	X

Bảng 10. So sánh chi phí các dạng biểu thức của cùng một hàm

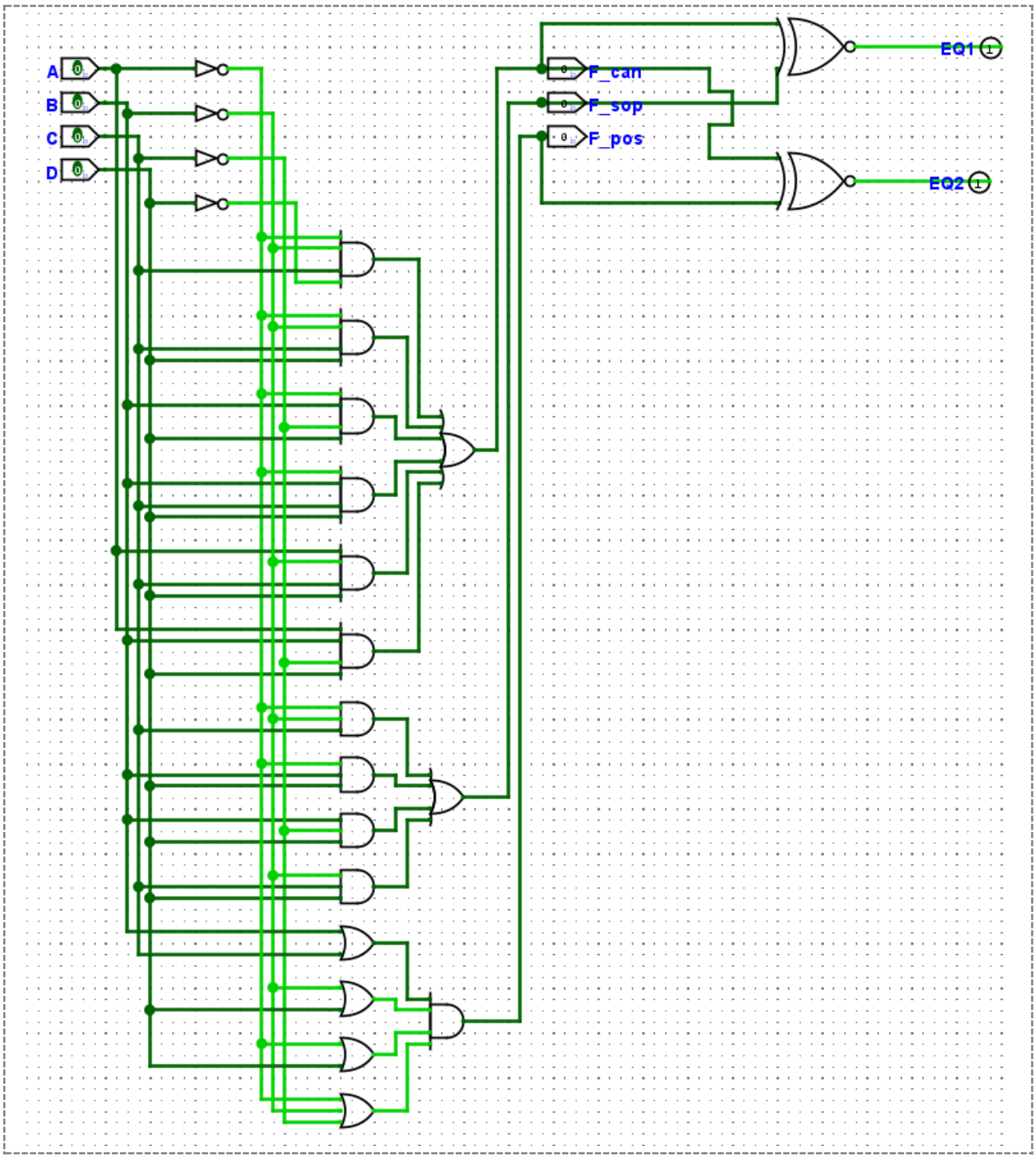
Dạng biểu thức	O(B)	Tổng literal	C(B)	Số cổng AND/OR (đếm trên Logisim)	Số cổng NOT
SoP chính tắc	6	24	30	1 OR6, 6 AND4	4
SoP tối giản	4	12	16	1 OR4, 4 AND3	4
PoS chính tắc	10	40	50	1 AND10, 10 OR4	4
PoS tối giản	4	9	13	1 AND4, 3 OR2, 1 OR3	4

Câu 4.3 (Đánh giá). Đối chiếu với dự đoán ở Bước 3.2. Điều gì quyết định biểu thức tối giản rẻ hay đắt: **số** ô 1/ô 0 trong bìa, hay **kích thước** các nhóm gom được? Giải thích bằng số liệu ở Bảng 10.

Trả lời:
- Điều quyết định biểu thức tối giản rẻ hay đắt là kích thước các nhóm gom được. Vì:
  + Dạng SoP: tối giản rẻ hơn chính tắc 2 giá trị ở O(B) – số nhóm gom được và rẻ hơn ở số lượng literal trong từng term (4 nhóm 3 so với 6 nhóm 4)
  + Dạng Pó: tối giản rẻ hơn chính tắc 6 giá trị ở O(B) – số nhóm gom được và rẻ hơn ở số lượng literal 3 OR 2, 1 OR 3 so với 10 OR 4

KIỂM CHỨNG Mở **Analyze Circuit ▸ Minimized** (nếu có) cho mạch F_can, và ghi lại biểu thức mà công cụ đưa ra.

Biểu thức tối giản do Logisim đưa ra	A’B’C+A’CD+BC’D+B’CD
Trùng với nghiệm SoP nào của Sinh viên? Chi phí có bằng không?	Trùng với nghiệm SoP ở bước 4.3. Chi phí bằng nhau


A
B
C
D
F_can
F_sop
F_pos
EQ1
EQ2

16 of 16 rows shown

A	B	C	D	EQ1	F_can	F_sop	F_pos	EQ2
0	0	0	0	1	0	0	0	1
0	0	0	1	1	0	0	0	1
0	0	1	0	1	1	1	1	1
0	0	1	1	1	1	1	1	1
0	1	0	0	1	0	0	0	1
0	1	0	1	1	1	1	1	1
0	1	1	0	1	0	0	0	1
0	1	1	1	1	1	1	1	1
1	0	0	0	1	0	0	0	1
1	0	0	1	1	0	0	0	1
1	0	1	0	1	0	0	0	1
1	0	1	1	1	1	1	1	1
1	1	0	0	1	0	0	0	1
1	1	0	1	1	1	1	1	1
1	1	1	0	1	0	0	0	1
1	1	1	1	1	0	0	0	1

THÍ NGHIỆM 5 (mở rộng) — Khi bìa Karnaugh "bó tay"

Thời lượng gợi ý: 20 phút · Circuit: **TN5** · Điểm: 1,0

> **Tình huống "Đèn hành lang 3 công tắc"**
> Một hành lang có ba công tắc **A, B, C** ở ba đầu. Gạt **bất kỳ** công tắc nào cũng làm đèn đổi trạng thái (đang sáng thì tắt, đang tắt thì sáng). Ban đầu cả ba công tắc ở vị trí 0 và đèn tắt. Ngõ ra F = 1 khi đèn sáng.

Bước 5.1 – Lập bảng chân trị. Gợi ý: đi theo thứ tự 000 → 001 → 011 → 010 → 110 → 111 → 101 → 100 (mỗi bước chỉ gạt **một** công tắc) và suy ra F.

Bảng 11. Bảng chân trị F(A, B, C)

A	B	C	F
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	0
1	1	0	0
1	1	1	1

Câu 5.1 (Hiểu). Vì sao thứ tự đi ở gợi ý lại tiện cho việc điền F? Nó giống quy tắc nào mà Sinh viên đã dùng khi đánh nhãn hàng và cột của bìa Karnaugh?

Trả lời:
- Thứ tự gợi ý giúp ta chỉ quản lý 01 bước thay đổi trạng thái từ bật sang tắt hoặc tắt sang bật. Khiến cho việc quản lý trạng thái dễ dàng hơn.
- Nó giống quy tắc đánh mã Gray mà sinh viên dùng khi đánh nhãn hàng và cột của bìa Karnaugh. Cụ thể là: 00 – 01 – 11 – 10 chứ không phải 00 – 01 – 10 – 11.

Bước 5.2 – Bìa Karnaugh 3 biến. Điền F vào bìa.

A \ BC	00	01	11	10
0		1		1
1	1		1	

Câu 5.2 (Phân tích). Có gom được nhóm từ 2 ô trở lên không? Giải thích bằng cách xét hai ô kề nhau (khác nhau đúng 1 bit ngõ vào) và giá trị F của chúng. Từ đó viết SoP tối giản và tính C(B).

Trả lời:
- Không gom nhóm từ 2 ô trở lên được. Lý do là vì khi 2 ô kề nhau, chúng chỉ khác nhau đúng 1 bit. Tức là trạng thái chuyển từ bật sang tắt hoặc tắt sang bật. Không thể tồn tại trường hợp thay đổi 1 bit mà đầu ra vẫn là 1 được vì đề bài đã nói rằng khi thay đổi 1 bit thì trạng thái của đầu ra bị thay đổi.

DỰNG MẠCH Trong circuit **TN5**, dựng F = A ⊕ B ⊕ C bằng **2 cổng XOR 2 ngõ vào** (dùng chung nhãn A, B, C, F). Kiểm tra với Bảng 11 (Poke hoặc Analyze Circuit).

Số hàng khớp với Bảng 11 (trên 8)	8/8

ĐÁNH GIÁ Thêm công tắc thứ tư, thứ năm… thì sao? Điền Bảng 12 (với n = 4, 5 hãy suy luận từ cấu trúc bìa/bảng chân trị, không cần vẽ hết).

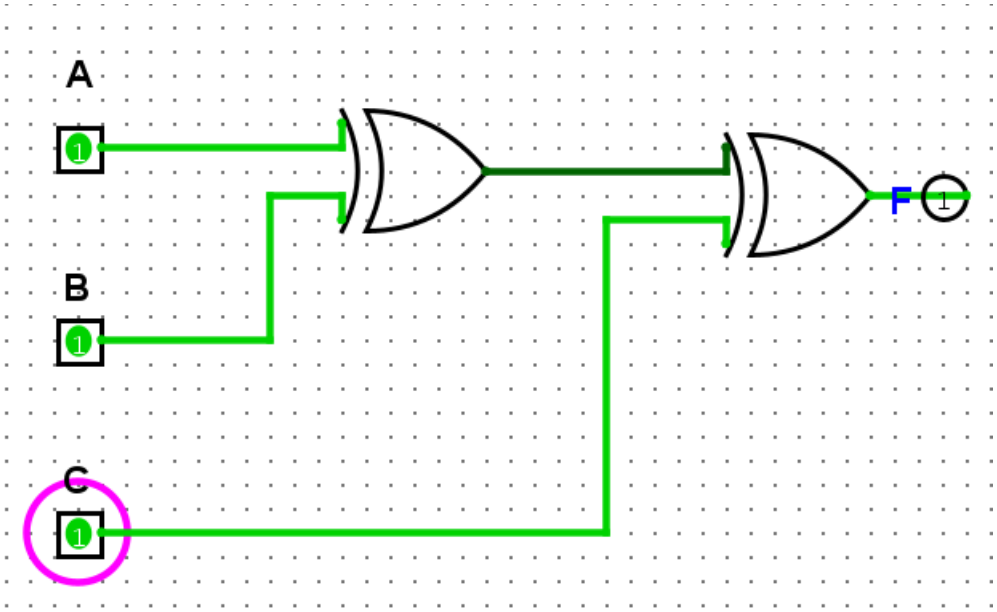
Bảng 12. SoP tối giản và XOR khi số công tắc tăng

Số công tắc n	Số term của SoP tối giản	Số cổng XOR 2 ngõ vào
3	4	2
4	8	3
5	16	4

Câu 5.3 (Đánh giá – câu hỏi mở). Công thức chi phí C(B) ở slide có phản ánh đúng chi phí của phương án XOR không? "Bìa Karnaugh không rút gọn được" có nghĩa là hàm không thể hiện thực gọn hơn không? Khi thiết kế, Sinh viên sẽ chọn phương án nào và dựa vào tiêu chí gì? (Không có một đáp án duy nhất; hãy nêu lập luận của Sinh viên.)

Trả lời:

- **Công thức $C(B)$ không phản ánh đúng chi phí của phương án XOR:** Công thức $C(B) = O(B) + \Sigma P_i(B)$ chỉ được xây dựng để đo đếm chi phí cho các cấu trúc logic 2 tầng chuẩn (AND/OR/NOT). Nó không đo được chi phí thực tế của các cổng phức hợp như XOR, vốn được cấu tạo từ nhiều cổng logic cơ bản bên trong (2 AND + 1 OR + 2 NOT cho mỗi cổng XOR 2 ngõ vào).
- **"Bìa Karnaugh không rút gọn được" không đồng nghĩa với việc mạch không thể hiện thực gọn hơn:** Việc bìa K-map không thể gom nhóm chỉ cho thấy hàm không thể tối giản hơn nếu bắt buộc biểu diễn dưới dạng 2 tầng AND/OR (SoP/PoS). Hàm vẫn hoàn toàn có thể hiện thực cực kỳ gọn nhẹ nếu thay đổi tập cổng cơ sở (Basis Gate) sang dùng cổng XOR/XNOR hoặc cấu trúc logic nhiều tầng.
- **Lựa chọn phương án và tiêu chí thiết kế:** Em chọn **phương án sử dụng cổng XOR** dựa trên các tiêu chí: tối ưu diện tích và số lượng linh kiện (chỉ tốn $n-1$ cổng XOR thay vì 2^{n-1} term của SoP), giảm độ phức tạp đi dây trên sơ đồ mạch, và đảm bảo tính mở rộng linh hoạt (Scalability) khi tăng số lượng công tắc n trong tương lai mà không cần thiết kế lại toàn bộ mạch.



a	b	c	F
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	0
1	1	0	0
1	1	1	1

TỔNG KẾT VÀ NỘP BÀI (15 phút)

Danh sách kiểm tra trước khi nộp

☑	Nội dung
☑	File **IT012_Lab01_<MSSV>.circ** có đủ 5 circuit **TN1, TN2, TN3, TN4, TN5**, mọi Input/Output Pin đều có nhãn.
☑	File báo cáo này đã điền đầy đủ các bảng, câu hỏi và dán ảnh chụp mạch vào các khung nét đứt.
☑	Mỗi mạch đã được kiểm chứng bằng Analyze Circuit (hoặc Poke) và số liệu ghi trong báo cáo khớp với mạch nộp.
☑	Đặt tên file báo cáo: **IT012_Lab01_<MSSV>_BaoCao**; nộp cả hai file theo hướng dẫn của giảng viên.

Phản tư

Câu R1. Từ Thí nghiệm 2, Sinh viên rút ra điều gì về sự khác nhau giữa **kiểm chứng bằng mô phỏng** và **chứng minh bằng đại số**? Nêu một tình huống nên dùng mỗi cách.

Trả lời:

- Sự khác nhau cốt lõi giữa kiểm chứng bằng mô phỏng và chứng minh bằng đại số là nằm ở yếu tố số lượng biến đầu vào. Cụ thể như sau:
  + Kiểm chứng bằng mô phỏng (máy tính) chỉ có thể chịu đựng vét cạn khi số lượng biến nằm ở mức ít, trung bình. Nếu số lượng biến tăng vọt, máy tính sẽ gặp tình trạng bùng nổ tổ hợp, dẫn đến thời gian tính toán tốn rất nhiều thời gian và thậm chí là phi thực tế.
  + Kiểm chứng bằng chứng minh bằng đại số sẽ được sử dụng dành cho những trường hợp số lượng biến lớn và cực lớn, hoặc khi cần chứng minh tổng quát đúng với mọi số lượng biến.

Câu R2. Từ Thí nghiệm 3, 4 và 5, Sinh viên sẽ cân nhắc những điều gì khi được giao tối ưu một hàm Boole mới (gom 1 hay gom 0, cách gom, loại cổng sử dụng…)?

Trả lời:

- Gom nhóm theo 1 hay 0 sẽ tùy vào số lượng số 0, 1 có trên bìa Karnaugh. Ví dụ có thể gom được nhiều nhóm 1 to hơn thì sẽ gom theo 1 và ngược lại. Nhằm tối ưu chi phí thiết kế.
- Đảm bảo các nhóm được gom theo quy tắc lũy thừa của 2, và đảm bảo rằng ưu tiên gom các nhóm lớn hơn.
- Không chỉ dùng AND OR NOT mà còn có thể tận dụng XOR, XNOR để tối ưu chi phí nhiều hơn nữa.

Câu R3. Nêu **một** lỗi hoặc nhầm lẫn Sinh viên đã gặp trong buổi thực hành, Sinh viên phát hiện ra bằng cách nào và bài học rút ra là gì?

Trả lời:

- Lỗi: Vẽ sai mạch số dẫn đến Analyze Circuit đưa ra đáp án không khớp với lý thuyết đại số Boole.
- Phát hiện bằng cách kiểm tra lại sơ đồ mạch trên logisim, vì lý thuyết đại số Boole chắc chắn đã đúng.
- Bài học rút ra chính là luôn lấy lý thuyết đại số Boole làm cơ sở để so sánh đáp án của mạch tự vẽ, từ đó nhận ra sớm lỗi sai và khắc phục.

Tài liệu tham khảo

- Slide bài giảng Chương 3 – Đại số Boole (IT012, Khoa Kỹ thuật Máy tính, UIT).
- Tài liệu *Hướng dẫn sử dụng Logisim* do giảng viên cung cấp.